

Теоретические основы информатики

Разделы	Темы
Информация и данные	Основные понятия информатики Кодирование информации
Двоичная математика	Системы счисления Битовые операции
Модели и структуры данных	Типы данных Структуры данных
Хранение и обработка данных	Структурированные файлы Парсинг данных
	Регулярные выражения
Методы информатики	Алгоритмизация Сортировка

Беляков Андрей Юрьевич

к.т.н., доцент

Тема 9

gitverse.ru/band/TOI

Раздел 4. Хранение и обработка данных

Тема 7

Плоские и иерархические данные

- Структурированные файлы форматов json, csv, yaml, xml.
- Библиотеки и методы работы со структурированными данными.

Тема 8

Парсинг данных

- Автоматизация сбора, фильтрации и агрегации данных.
- Автоматизация обработки данных (запись, чтение, библиотеки).

Тема 9

Регулярные выражения

- Понятие, области применения, синтаксис.
- Практические примеры поиска и замены в строках.

Тема 9

Регулярные выражения

<https://regex101.com/>

Вопрос 1:

Понятие, области применения, синтаксис

Вопрос 2:

Практические примеры поиска и замены в строках

Понятие, области применения, синтаксис



Регулярные выражения - это специальные текстовые строки, которые используются в качестве шаблонов для сопоставления с наборами других строк

Области применения:

- поиск и замена в текстовых редакторах
- валидация данных (email, телефон, пароль)
- парсинг HTML/логов
- в утилитах UNIX (grep, sed, awk)

Regular expression

- формальный язык поиска и манипуляций со строками

Стивен Клини - американский математик, в 1956 г. ввёл понятие регулярных выражений, разработал алгебру Клини с базовой операцией звёздочка Клини (* - 0 или более раз), является создателем формальной теории регулярных выражений.

- истоки лежат в теории формальных языков - 1940-ые годы
- алгебра Клини в 1950-х с операциями выбора, итерации (*), конкатенации
- в 1960-х встраивание `re` в редактор QED, появились символы `^` и `$` для начала и конца строки
- в 1970-х внедрение `re` утилит ОС в UNIX (`grep`, `sed`, `awk`)
- в 1980-х стандартизация `re`
- с 1990-х поддержка `re` стала стандартом для языков программирования

- два подхода к реализации функционала `re`

автоматный (Java, JavaScript) - гарантирует линейное время, быстрый, не поддерживает обратные ссылки

рекурсивный (Python, Perl, Notepad++) - поддерживает обратные ссылки, гибкий, медленнее, может зависать

обратные ссылки (Backreferences), опережающие/ретроспективные проверки (lookahead/lookbehind), группы могут приводить к catastrophic backtracking – время поиска растёт экспоненциально

RegExr – базовый синтаксис

Литерал Literal	символ (последовательность символов) определяет точное совпадение
Метасимвол Metacharacter	символ определяет логику поиска
Граница Anchor	метасимвол, который привязывает паттерн к началу, концу или границе слова
Символьный класс Character Class	набор символов, заключённых в квадратные скобки [...], искать совпадения только из этого набора
Квантификатор Quantifier	метасимвол, который указывает, сколько раз должен повторяться предыдущий элемент жадность, ленивость
Группа Group	часть выражения, заключённая в круглые скобки (...), извлекать и переиспользовать именованные, захватываемые, не захватываемые
Обратная ссылка Backreference	ссылка на значение, захваченное ранее в выражении

Литералы

<https://regex101.com/>

Литерал - это символ или последовательность символов, обозначают сами себя

Регулярное Выражение

Совпадения · 6 время · 200 μs ⓘ

: r" строка

" gi 📋

Тестовая Строка ● +

1 строка · рок ↵

2 строка · Строка · сорока · строка · ↵

3 отрок · строки · острока · строкандия · |

Пояснение ^

Информация О Соответствиях ▾

Соответствие 1	0-6	строка
Соответствие 2	11-17	строка
Соответствие 3	18-24	Строка
Соответствие 4	32-38	строка
Соответствие 5	54-60	строка
Соответствие 6	61-67	строка

Метасимволы

Метасимвол управляет логикой поиска

Метасимвол	Название	Значение
.	Точка	Любой символ (кроме новой строки)
^	Крышка	Начало строки
\$	Доллар	Конец строки
*	Звёздочка	0 или более повторений
+	Плюс	1 или более повторений
?	Вопрос	0 или 1 повторение
\	Бэкслеш	Экранирование (отмена спец. значения)
	Вертикальная черта	ИЛИ (альтернатива)
()	Круглые скобки	Группировка
[]	Квадратные скобки	Символьный класс

Метасимволы

Метасимвол управляет логикой поиска

Regex: `st.l`

Тестовая Строка

- 1 стол*стул*устал*стал*
- 2 Стамбул*сталь

Regex: `st.?l`

Тестовая Строка

- 1 стол*стул*устал*стал*
- 2 Стамбул*сталь*стл

Regex: `st[ou]l`

Тестовая Строка

- 1 стол*стул*устал*стал*
- 2 Стамбул*сталь*стл

Regex: `st[oua]l`

Тестовая Строка

- 1 стол*стул*устал*стал*
- 2 Стамбул*сталь*стл

Странно: почему не Стамбул, стал, стОл, стол, стул, ...

Regex: `st.+l`

Тестовая Строка

- 1 стол*стул*устал*стал*
- 2 Стамбул*сталь*стл*стОл*
- 3 столллллл

Информация О Соответствиях

Соответствие 1	0-20	стол*стул*устал*стал
Соответствие 2	22-44	Стамбул*сталь*стл*стОл
Соответствие 3	45-54	столллллл

Захватывает строки целиком

Regex: `st[a-я]*l`

Тестовая Строка

- 1 стол*стул*устал*стал*
- 2 Стамбул*сталь*стл

- далее обсудим жадность квантификаторов

Квантификаторы

- это метасимволы, которые определяют сколько раз должен повторяться предыдущий элемент

Квантификатор	Значение
*	0 или более раз
+	1 или более раз
?	0 или 1 раз
{n}	ровно n раз
{n,}	n или более раз
{n,m}	от n до m раз

квантификаторы по умолчанию **жадные**

- захватывают максимально длинную подстроку

квантификатор можно сделать **ленивым**

- будет захватывать минимально возможную подстроку

- добавить **?** после квантификатора

greedy

Regex: `<b>.*</b>`

Test String: `<b>hello</b>.*<b>world</b>`

Match 1: `<b>hello</b>.*<b>world</b>`

Information about matches: `<b>hello</b>.*<b>world</b>`

lazy

Regex: `<b>.*?</b>`

Test String: `<b>hello</b>.*<b>world</b>`

Match 1: `<b>hello</b>`

Match 2: `<b>world</b>`

Information about matches: `<b>hello</b>`, `<b>world</b>`

для парсинга пригодится

Квантификаторы

отмена жадности

greedy

`: r" ст.+л`

`" gim`

Тестовая Строка

1

стол*стул*устал*стал*

2

Стамбул*сталь*стл*стл

3

столл

Информация О Соответствиях

Соответствие 1	0-20	стол*стул*устал*стал
Соответствие 2	22-44	Стамбул*сталь*стл*стл
Соответствие 3	45-54	столл

было

lazy

`: r" ст.+?л`

`" gim`

Тестовая Строка

1

стол*стул*устал*стал*

2

Стамбул*сталь*стл*стл

3

столл

Информация О Соответствиях

Соответствие 1	0-4	стол
Соответствие 2	5-9	стул
Соответствие 3	11-15	стал
Соответствие 4	16-20	стал
Соответствие 5	22-29	Стамбул
Соответствие 6	30-34	стал
Соответствие 7	36-44	стл*стл
Соответствие 8	45-49	стол

стало

Границы

- это метасимволы, которые привязывают паттерн к границам строки или слова

`: r" \b строка`

Тестовая Строка

```
1 строка*рок␣
2 строка*Строка*сорока*строка*␣
3 отрок*строки*острока*строкандия*␣
4 острокан*Строка␣
5 строка
```

`: r" строка$`

Тестовая Строка

```
1 строка*рок␣
2 строка*Строка*сорока*строка*...␣
3 отрок*строки*острока*строкандия*␣
4 острокан*Строка␣
5 строка
```

Якорь	Значение	Влияние <code>re.MULTILINE</code>
<code>^</code>	Начало строки	Да - начало каждой строки
<code>\$</code>	Конец строки	Да - конец каждой строки
<code>\A</code> <code>\Z</code>	Начало всего текста Конец всего текста	Нет - только начало всего текста Нет - только конец всего текста
<code>\b</code> <code>\B</code>	Граница слова НЕ граница слова	

`: r" ^ строка`

Тестовая Строка

```
1 строка*рок␣
2 строка*Строка*сорока*строка*␣
3 отрок*строки*острока*строкандия*␣
4 острокан*Строка␣
5 строка
```

`: r" \b строка \b`

" gim

Тестовая Строка

```
1 строка*рок␣
2 строка*Строка*сорока*строка*␣
3 отрок*строки*острока*строкандия*␣
4 острокан*Строка␣
5 строка
```

Классы

Регулярка	Совпадает
[aeiou]	Любая гласная буква (a, e, i, o, u)
[0-9]	Любая цифра от 0 до 9
[a-z]	Любая строчная буква латиницы
[A-Z]	Любая заглавная буква латиницы
[a-zA-Z]	Любая латинская буква (регистр не важен)

Регулярка	Совпадает
[^0-9]	Любой символ, кроме цифры
[^aeiou]	Любой символ, кроме гласных
[^a-z]	Любой символ, кроме строчных букв латиницы

отрицание в символьном классе

класс - набор символов, который задаёт множество
допустимых символов в позиции

Шорткат	Эквивалент	Значение
\d	[0-9]	Любая цифра
\D	[^0-9]	Любая не цифра
\w	[a-zA-Z0-9_]	Буква, цифра или подчёркивание
\W	[^a-zA-Z0-9_]	Не буква, цифра и не подчёркивание
\s	[\t\n\r\f\v]	Пробельный символ
\S	[^ \t\n\r\f\v]	Не пробельный символ

предопределённые классы

Классы

`: r" [^\s\d=]+" gim`

Тестовая Строка

```
1 стол*123*стул*7099*high*  
2 length*==*666*  
3 1024*метров*9577*....
```

Информация О Соответствиях

Соответствие 1	0-4	стол
Соответствие 2	9-13	стул
Соответствие 3	19-23	high
Соответствие 4	25-31	length
Соответствие 5	43-49	метров

`: r" [2-9]{3,}" gim`

Тестовая Строка

```
1 стол*123*стул*7099*стал*  
2 длина*==*666*  
3 1024*метров*9577*....
```

Информация О Соответствиях

Соответствие 1	33-36	666
Соответствие 2	49-53	9577

`: r" [abc]+" gim`

Тестовая Строка

```
1 a*bb*cccccc*cac*beb*
```

Информация О Соответствиях

Соответствие 1	0-1	a
Соответствие 2	2-4	bb
Соответствие 3	5-12	cccccc
Соответствие 4	13-16	cac
Соответствие 5	17-18	b
Соответствие 6	19-20	b

`: r" \w+" gim`

Тестовая Строка

```
1 стол*стул*устал*стал*  
2 Стамбул*сталь*стл*стол*  
3 столлллл
```

Информация О Соответствиях

Соответствие 1	0-4	стол
Соответствие 2	5-9	стул
Соответствие 3	10-15	устал
Соответствие 4	16-20	стал
Соответствие 5	22-29	Стамбул
Соответствие 6	30-35	сталь
Соответствие 7	36-39	стл
Соответствие 8	40-44	стол
Соответствие 9	45-54	столлллл

для Python режим Unicode
(для кириллицы)
включён по умолчанию

`: r" \d+" gim`

Тестовая Строка

```
1 стол*1*2*3*стул*устал*стал*  
2 длина*==*666*  
3 1024*метров
```

Информация О Соответствиях

Соответствие 1	5-6	1
Соответствие 2	7-8	2
Соответствие 3	9-10	3
Соответствие 4	36-39	666
Соответствие 5	40-44	1024

Группы

- для извлечения или замены данных

Regex: `:\r" (\d{3})[-\.](\d+)` " gim

Тестовая Строка

```
1 123-456
2 12.999
3 45*707
4 912-454545
5 404
6
```

Информация О Соответствиях

Соответствие 1	0-7	123-456
Группа 1	0-3	123
Группа 2	4-7	456
Соответствие 2	22-32	912-454545
Группа 1	22-25	912
Группа 2	26-32	454545

Regex: `:\r" (\d{3})[-\.](?:\d+)` " gim

Тестовая Строка

```
1 123-456
2 123.999
3 45*707
4 912-454545
```

Информация О Соответствиях

Соответствие 1	0-7	123-456
Группа 1	0-3	123
Соответствие 2	8-15	123.999
Группа 1	8-11	123
Соответствие 3	23-33	912-454545
Группа 1	23-26	912

(?:) - это незахватывающая группа

Regex: `:\r" ([a-я]+)-([a-я]+)` " gim

Тестовая Строка

```
1 кока-кола
```

Соответствие 1 0-9 кока-кола

Группа 1 0-4 кока

Группа 2 5-9 кола

Подстановка Замены · 1 время · 300 μs

Regex: `\g<2>-\g<1>`

```
1 кола-кока
```

Обратные ссылки

- для сравнения с тем, что уже было

```
regex: r" \b(\w+)-\1\b " gimu
```

Тестовая Строка

1 гори-гори*моя*звезда

Информация О Соответствиях

Соответствие 1	0-9	гори-гори
Группа 1	0-4	гори

по номеру
группы

```
regex: r" \b(?P<first>\w+)-(?P=first)\b " gim
```

Тестовая Строка

1 гори-гори*моя*звезда
2 сколько-сколько*мне*осталось?
3 тук-тук,*послышалось*из-за
4 как*так
5 тук*тук

Информация О Соответствиях

Соответствие 1	0-9	гори-гори
Группа first	0-4	гори
Соответствие 2	21-36	сколько-сколько
Группа first	21-28	сколько
Соответствие 3	51-58	тук-тук
Группа first	51-54	тук

по имени
группы

Контрольные вопросы

- понятие регулярное выражение
- области применения регулярных выражений
- что такое литерал и метасимвол
- классы символов
- квантификаторы (жадность и ленивость)
- группы символов



Вопрос 2

Практические примеры поиска и замены в строках



- особенности реализации теории регулярок на практике
- на примерах реализации в Python
- какая библиотека и способы её использования
- какие есть методы
- как именно применять синтаксис регуляро

Литералы

Литерал - это символ или последовательность символов, обозначают сами себя

Два способа использования регулярок:

метод скомпилированного паттерна или функция модуля re

```
import re
```

```
txt = "000abc000"
```

```
ptn = re.compile(r'abc')  
m = ptn.search(txt)  
print(m)
```

```
m = re.search(r'abc', txt)  
print(m)
```

метод объекта ptn

функция модуля re

эквивалентны по функциональности

```
<re.Match object; span=(3, 6), match='abc'>
```

Литералы

```
import re
```

```
txt = "000abc000"
```

```
ptn = re.compile(r'abc')
m = ptn.search(txt)

if m:
    print(m.group()) # abc
    print(m.span()) # (3, 6)
    print(m.start()) # 3
    print(m.end()) # 6
```

метод объекта ptn

```
m = re.search(r'abc', txt, re.I)
```

```
if m:
    print(m.group()) # abc
    print(m.span()) # (3, 6)
    print(m.start()) # 3
    print(m.end()) # 6
```

функция модуля re

эквивалентны по функциональности

Метод паттерна или функция модуля

```
# Функция: флаги можно менять при каждом вызове
re.search(r'abc', 'ABC')
re.search(r'abc', 'ABC', re.IGNORECASE)
```

```
# Компиляция: флаги фиксируются
compiled = re.compile(r'abc', re.IGNORECASE)
compiled.search('ABC') # флаг уже внутри
```

- в функции `re.search()` флаги передаются как аргумент
- в скомпилированном объекте флаги задаются один раз при компиляции и не могут быть изменены при вызове метода

```
def search(pattern, text, flags=0):
    compiled = re.compile(pattern, flags)
    return compiled.search(text)
```

- Python каждый раз делает замену RunTime
- функции модуля `re` неявно компилируются в паттерн

Флаги регулярнок (модификаторы)

- если флагов несколько, то соединяем через ИЛИ

```
re.search(r'abc', 'ABC', re.I | re.M)
re.search(r'abc', 'ABC', re.IGNORECASE | re.MULTILINE)

pattern = re.compile(r'abc', re.IGNORECASE | re.MULTILINE)
pattern.search('ABC')
```

Флаг	Значение
re.IGNORECASE (re.I)	Игнорирует регистр
re.MULTILINE (re.M)	^ и \$ работают для каждой строки
re.DOTALL (re.S)	Точка . совпадает с \n
re.VERBOSE (re.X)	Разрешает комментарии и пробелы в паттерне
re.ASCII (re.A)	\w, \d, \s работают только с ASCII

Методы и функции в Python

Функция модуля re	Метод объекта
<code>re.search(pattern, text, flags)</code>	<code>pattern.search(text)</code>
<code>re.match(pattern, text, flags)</code>	<code>pattern.match(text)</code>
<code>re.fullmatch(pattern, text, flags)</code>	<code>pattern.fullmatch(text)</code>
<code>re.findall(pattern, text, flags)</code>	<code>pattern.findall(text)</code>
<code>re.finditer(pattern, text, flags)</code>	<code>pattern.finditer(text)</code>
<code>re.sub(pattern, repl, text, count, flags)</code>	<code>pattern.sub(repl, text, count)</code>
<code>re.subn(pattern, repl, text, count, flags)</code>	<code>pattern.subn(repl, text, count)</code>
<code>re.split(pattern, text, maxsplit, flags)</code>	<code>pattern.split(text, maxsplit)</code>

Функционал в Python

Метод	Возвращает	Пример
<code>.search()</code>	первое совпадение	<code>comp.search(text)</code>
<code>.match()</code>	совпадение в начале	<code>comp.match(text)</code>
<code>.fullmatch()</code>	вся строка совпадает	<code>comp.fullmatch(text)</code>
<code>.findall()</code>	список вхождений	<code>comp.findall(text)</code>
<code>.finditer()</code>	итератор вхождений	<code>comp.finditer(text)</code>
<code>.sub()</code>	строка с заменами	<code>comp.sub(repl, text)</code>
<code>.subn()</code>	Строка и кол-во замен	<code>comp.subn(repl, text)</code>

Литералы

```
comp = re.compile(r'abc')
```

```
matches = comp.findall('ABC-abc-abc-')  
if matches:  
    for match in matches:  
        print(match)
```

```
abc  
abc
```

```
matches = comp.finditer('ABC-abc-abc-')  
if matches:  
    for match in matches:  
        print(match.group(), match.span())
```

```
abc (4, 7)  
abc (8, 11)
```

Метасимволы

```
import re

lst = ['ct', 'cat', 'cot', 'caaaat', 'cart', 'c.t']

for s in lst: print(re.search(r'c.t', s))

for s in lst: print(re.search(r'c..t', s))

for s in lst: print(re.search(r'ca*t', s))

for s in lst: print(re.search(r'c.*t', s))

for s in lst: print(re.search(r'ca+t', s))

for s in lst: print(re.search(r'ca?t', s))
```

примеры использования
метасимволов

[обсудить](#)

Метасимволы

экранирование

```
import re

lst = [
    'ct', 'cat',
    'cot', 'caaaat',
    'cart', 'c.t'
]

for s in lst:
    print(re.search(r'c\.t', s))

for s in lst:
    print(re.search('c\\.t', s))
```

если не использовать сырые строки, то
следует использовать двойной обратный слеш \\

обсудить

экранирование

когда требуется найти сам метасимвол

Что ищем		Регулярка
Точку	.	\.
Звёздочку	*	*
Знак вопроса	?	\?
Обратный слэш	\	\\
Круглую скобку	(	\(

Границы

```
re.search(r'^Python', 'Python is lang...')
re.search(r'Python!$', '... Python!')
re.search(r'^Python$', 'Python')
```

```
re.search(r'\bPython\b', '... Python ...')
re.search(r'\bPython\b', '... Python! ...')
```

```
re.search(r'\Bth\B', '... then ...')
re.search(r'\Bth\B', '... Python ...')
```

```
re.search(r'\bкот\B', ', который ...')
re.search(r'\bкот\b', '... Кот ...', re.I)
```

Границы - это метасимволы, которые привязывают паттерн к границам строки или слова

```
text = "Кот, собака и 2 пtiцы."
words = re.findall(r'\w+', text)
# ['Кот', 'собака', 'и', '2', 'птицы']

# найти слова только из латиницы
text = "Hello, мир - Python!"
re.findall(r'\w+', text, re.ASCII)
# ['Hello', 'Python']
```

`\w` - СИМВОЛ ДЛЯ СЛОВ

- режим ASCII - `re.ASCII`
- если установить этот флаг, то `\w` будет включать только латинские буквы [a-zA-Z], цифры [0-9] и символ подчеркивания `_`
- кириллица перестанет считаться "буквой"

```
<div>
  <a href="https://example.com">Главная</a>
  <a href="/about">О нас</a>
  <a href="/contact">Контакты</a>
  <span>Не ссылка</span>
</div>
```

Группы Классы Квантификаторы

```
comp = re.compile(r'(<a\s+href="([^\"]+)"\s*>(.*?)</a>') , re.I)
```



```
matches = comp.findall(html)
for tag, href, text in matches:
    print(tag)
    print(f"Текст: {text.strip():<10} → Ссылка: {href}")
```

используем вложенные группы

```
for match in comp.finditer(html):
    tag = match.group(1)      # первая группа: ссылка
    href = match.group(2)     # первая группа: ссылка
    text = match.group(3)     # вторая группа: текст
    print(tag)
    print(f"Текст: {text.strip():<10} → Ссылка: {href}")
```

```
<a href="https://example.com">Главная</a>
Текст: Главная      → Ссылка: https://example.com
<a href="/about">О нас</a>
Текст: О нас        → Ссылка: /about
<a href="/contact">Контакты</a>
Текст: Контакты     → Ссылка: /contact
```

Обратные ссылки

```
text = "hello-hello world-world test-test"
```

```
comp = re.compile(r'\b(\w+)-\1\b', re.I)
matches = comp.findall(text)
print(matches) # ['hello', 'world', 'test']
```

```
comp = re.compile(r'\b(?P<first>\w+)-(?P=first)\b', re.I)
matches = comp.findall(text)
print(matches) # ['hello', 'world', 'test']
```

находим повторы
через группы
нумерованные и
именованные

```
text = "тик-так тик-ток тук-тук"
```

```
comp = re.compile(r'\b(\w+)-(\w+)\b', re.I)
result = comp.sub(r'\2-\1', text)
print(result) # так-тик ток-тик тук-тук
```

меняем местами группы:
`\2 - \1`

```
comp = re.compile(r'\b(\w+)-(\w+)\b', re.I)
result = comp.sub(r'\2-\1', text, count=1)
print(result) # так-тик тик-ток тук-тук
```

меняем местами
только первое совпадение

```
pattern = re.compile(r'\b(?P<first>\w+)-(?P<second>\w+)\b', re.I)
result = pattern.sub(r'\g<second>-\g<first>', text)
print(result) # так-тик ток-тик тук-тук
```

меняем местами
именованные группы

Regular expression

редкие проблемы

При рекурсивной реализации движка регулярок:

- обратные ссылки (Backreferences), группы, опережающие/ретроспективные проверки (lookahead/lookbehind), могут приводить к catastrophic backtracking, время поиска быстро растёт
- движок перебирает все возможные способы разбить n букв a на группы
- отсутствие совпадения становится причиной экспоненциального роста времени

```
import re, time

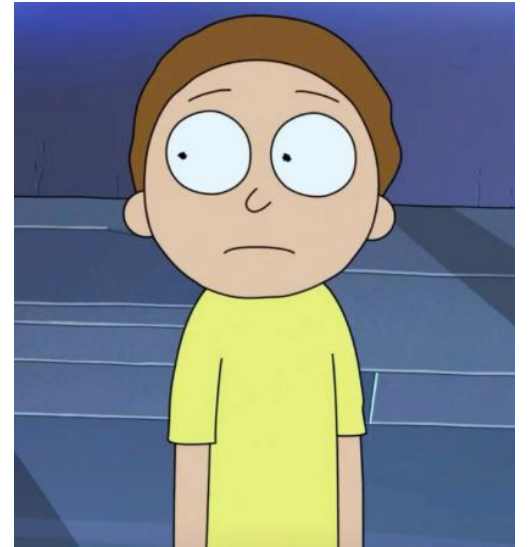
pattern = re.compile(r'^(a+)+$')

for n in range(10, 31, 2):
    text = "a" * n + "b"
    start = time.perf_counter()
    pattern.search(text) # нет совпадений
    elapsed = time.perf_counter() - start
    print(f"n = {n:2d}, время: {elapsed:.6f} сек")
```

n = 10,	время:	0.000060	сек
n = 12,	время:	0.000406	сек
n = 14,	время:	0.001380	сек
n = 16,	время:	0.003122	сек
n = 18,	время:	0.011630	сек
n = 20,	время:	0.046058	сек
n = 22,	время:	0.194298	сек
n = 24,	время:	0.724165	сек
n = 26,	время:	2.926202	сек
n = 28,	время:	11.524624	сек
n = 30,	время:	46.333280	сек

Контрольные вопросы

- библиотека в Python для подключения функционала регулярок ?
- какие два способа использования регулярок в Python ?
- основные методы/функции регулярок в Python ?
- флаги (модификаторы)
- как использовать символьные классы
- инвертированный символьный класс
- особенности использования обратных ссылок



Теоретические основы информатики

Раздел 4. Хранение и обработка данных

Тема 9: Регулярные выражения
